校园级 RAG
知识库平台环境安装与首次运行手册

适用对象：项目组全体成员 | Windows 10 / 11 | Java 17 + Python 3.11 + Vue3

手册定位	本文只说明校园级 RAG 知识库平台的本地开发、最小启动、RAG 联调和完整服务环境。项目采用 Java 微服务 + Python AI 运行时 + Vue3 前端，RAG 链路包含文档解析、分块、向量化、Elasticsearch 混合检索、Rerank、引用生成和版本过滤。

阅读方式

第一次配置先完成 Java、Python、Node.js 和 Docker 基础环境，再选择“最小联调”或“完整平台”启动。向量模型、LLM Key 和生产密钥不要提交到 Git。

目录

1. 系统环境总览
2. 版本、服务与资源清单
3. Windows 软件安装
4. 获取 Monorepo 代码与目录结构
5. 配置 Java 平台侧
6. 配置 Python RAG 运行时
7. 配置 Vue 前端与最小联调
8. Docker Compose 完整环境
9. RAG 链路验收与排障

附录 A 常见问题排查

1. 系统环境总览

校园级 RAG 知识库平台采用 Java 微服务承载用户、文档、检索、管理和评测，Python 运行时负责 LangGraph 编排、MCP 工具、RAG
检索/重排/生成和护栏。文档上传后进入对象存储，经过分块和向量化后写入
Elasticsearch；问答时进行关键词 + 向量混合检索、版本过滤、Rerank 和引用生成。

层次	组件	作用	首次安装
Java 平台	JDK 17、Maven 3.9+、Spring Boot 3.2.x	运行网关、认证、文档、检索、管理和评测服务	必须
微服务基础	Spring Cloud Alibaba 2022.0.x、Nacos 2.3.x、Gateway、OpenFeign	服务注册、配置、网关和服务间调用	最小联调可简化
数据与中间件	MySQL 8、Redis 7、RocketMQ 5、MinIO、Elasticsearch 8	元数据、缓存、异步消息、文件和混合检索	完整联调必须
Python AI	Python 3.11、LangGraph 0.2+、LangChain 0.3+、MCP	RAG 编排、工具调用、护栏和评测	必须
向量与模型	bge-large-zh v1.5、本地 Ollama 或 vLLM	生成 768 维中文向量	问答联调必须
前端	Vue 3.4+、Vite、TypeScript、Element Plus	问答、引用卡片、办事指南和管理端	必须
运维增强	Docker 24+、Nginx、K8s、SkyWalking 9.x	容器部署、反代、编排和链路追踪	可选

2. 版本、服务与资源清单

平台服务较多，建议先启动“最小联调集”：MySQL、Redis、Elasticsearch、MinIO、Nacos、Java gateway/search/doc 服务、Python rag-runtime 和前端。RocketMQ、Sentinel、Seata、SkyWalking 可在主链路稳定后加入。

组件	项目版本	默认端口或地址	用途
JDK / Maven	17 / 3.9+	-	Java 编译和构建
Python	3.11	-	RAG Runtime 和评测脚本
Node.js	18+	5173	Vue3 前端和 Vite
MySQL	8.0	3306	用户、文档、分块、查询日志和评测
Redis	7.x	6379	缓存、Token、限流和重建锁
Nacos	2.3.x	8848	注册中心和配置中心
RocketMQ	5.x	9876 NameServer	文档上传、查询日志和评测异步消息
Elasticsearch	8.x	9200	IK 关键词 + kNN 向量混合检索
MinIO	RELEASE.2024	9000/9001	PDF 原文、输出报告和评测文件
Python RAG	FastAPI 常见端口 8000	8000	LangGraph、MCP、RAG 和护栏

3. Windows 软件安装

1 安装 VS Code 与 Git：安装 VS Code 和 Git for Windows，Git PATH 选项选择 Git from the command line and also from 3rd-party software。

2 安装 JDK 17：安装 OpenJDK 17 或 Eclipse Temurin 17，并验证：

命令示例

```
java -version
javac -version
```

3 安装 Python 3.11：安装 Python 3.11，勾选 Add Python.exe to PATH；验证：

命令示例

```
python --version
pip --version
```

4 安装 Node.js：安装 Node.js LTS（项目要求 18+），验证：

命令示例

```
node -v
npm -v
```

5 安装 Docker Desktop：完整环境使用 Docker 24+ 和 Compose v2，确认 Engine running。

命令示例

```
docker --version
docker compose version
```

6 准备模型工具：安装 Ollama 或由负责人提供 vLLM 地址；bge-large-zh 用于中文向量化，LLM 可使用本地模型或团队统一 API。

4. 获取 Monorepo 代码与目录结构

1 设置 Git 身份：每名成员设置自己的姓名和邮箱。

命令示例

```
git config --global user.name "    "
git config --global user.email "    "
```

2 克隆仓库：在 D:\Projects 打开 PowerShell，把项目负责人提供的地址替换到命令中。

命令示例

```
git clone https://      /  /campus-rag-platform.git
cd campus-rag-platform
```

3 打开项目：执行：

命令示例

```
code .
```

命令示例

```
campus-rag-platform/
├─ pom.xml                 #   POM
├─ gateway-service/        #       JWT
├─ auth-service/           #
├─ doc-service/            #
├─ search-service/         # ES     +        Rerank
├─ admin-service/          #
├─ eval-service/           #      Agent
├─ rag-runtime/            # Python LangGraph MCP RAG
│  ├─ graph/               # StateGraph   Agent
│  ├─ mcp_server/          # search_regulation get_clause
│  ├─ rag/                 # indexer retriever faithful
│  ├─ guardrails/          #
│  └─ eval/                #
├─ frontend/               # Vue3 + Vite + TypeScript
├─ deploy/                 # Dockerfile Compose Nginx K8s
└─ docs/                   #
```

不要提交 .env、LLM Key、模型文件、MySQL 数据、MinIO 数据、Elasticsearch 数据、node_modules 和 Python 虚拟环境。

5. 配置 Java 平台侧

Java 侧至少包含网关、认证、文档、检索、管理和评测服务。服务注册和路由依赖 Nacos；文档检索依赖 MySQL、MinIO 和 Elasticsearch。

1 检查父 POM：确认 Java 版本为 17，Spring Boot 为 3.2.x，Spring Cloud Alibaba 为 2022.0.x。

2 构建所有 Java 服务：在仓库根目录执行：

命令示例

```
mvn -B -DskipTests package
```

3 启动 Nacos：使用项目 Compose 或 Nacos 安装包启动；控制台通常为 http://localhost:8848/nacos。

4 启动服务：按 README 或 deploy 编排顺序启动 gateway、auth、doc、search、admin、eval。

5 检查 Actuator：依次访问各服务的 /actuator/health，确认注册和配置加载正常。

如果项目提供 mvnw 或 mvnw.cmd，优先使用 Wrapper；如果 Nacos、MySQL、Redis 和 ES 由 Docker 启动，不要重复在本机安装同一端口的服务。

6. 配置 Python RAG 运行时

Python 运行时负责问题理解、混合检索工具调用、Rerank、引用生成、低置信拒答、敏感信息过滤和评测。建议为 rag-runtime 单独创建虚拟环境。

1 进入运行时目录：执行：

命令示例

```
cd campus-rag-platform\rag-runtime
```

2 创建虚拟环境：执行：

命令示例

```
python -m venv .venv
.venv\Scripts\Activate.ps1
```

3 安装依赖：如果仓库提供 pyproject.toml，按项目 README 安装；常见命令为：

命令示例

```
python -m pip install --upgrade pip
pip install -e .
```

4 配置服务地址：创建 .env 或复制模板，填写 JAVA_BASE、SEARCH_SERVICE_URL、DOC_SERVICE_URL、向量模型地址和 LLM 地址。不要提交 .env。

5 启动 Python 服务：常见 FastAPI 启动方式为：

命令示例

```
uvicorn src.api:app --host 0.0.0.0 --port 8000
```

6 检查工具链：确认 MCP Server 能调用 search_regulation、get_clause、list_versions 和 check_expiry；确认 embedding 维度为 768。

7. 配置 Vue 前端与最小联调

1 进入前端目录：执行：

命令示例

```
cd campus-rag-platform\frontend
```

2 安装依赖：执行：

命令示例

```
npm install
```

3 配置 API 地址：检查 vite.config.ts 和环境变量，确认 /api 请求代理到 gateway；如果本地网关为 8080，前端代理目标应一致。

4 启动前端：执行：

命令示例

```
npm run dev
```

5 验证问答页面：浏览器打开 Vite 地址，登录后提交“国家奖学金申请条件”等测试问题，检查回答是否带 [C:n] 引用和引用卡片。

最小联调顺序：MySQL → Redis → Elasticsearch → MinIO → Nacos → Java gateway/doc/search → Python rag-runtime → frontend。RocketMQ
未启动时，文档上传后的异步索引可能不可用，应先使用已有索引验证问答链路。

8. Docker Compose 完整环境

完整环境使用 Docker Compose 启动基础设施和部分服务。启动前必须准备强密码、JWT 密钥、模型地址和本地卷目录。

1 准备环境变量：复制项目模板：

命令示例

```
copy .env.example .env
```

2 填写关键变量：至少填写 DB 密码、Redis 密码、APP/JWT 密钥、MinIO 账号、Elasticsearch 地址和模型服务地址。

3 检查 Compose：执行：

命令示例

```
docker compose -f deploy/docker-compose.yml config
```

4 启动基础设施：执行：

命令示例

```
docker compose -f deploy/docker-compose.yml up -d mysql redis elasticsearch minio nacos
```

5 启动应用服务：基础设施健康后执行：

命令示例

```
docker compose -f deploy/docker-compose.yml up -d --build
```

6 查看状态与日志：执行：

命令示例

```
docker compose -f deploy/docker-compose.yml ps
docker compose -f deploy/docker-compose.yml logs -f search-service
```

7 访问前端或网关：按 compose 端口访问 gateway 和 frontend；检查 /actuator/health 或项目约定健康接口。

首次启动 ES、MinIO 和 Nacos 需要等待健康检查。文档上传后如果两分钟内没有 doc_chunk 索引，先查看 RocketMQ、search-service 和死信队列。

9. RAG 链路验收与排障

9.1 验收命令

命令示例

```
mvn test -pl search-service
python rag-runtime/eval/run_eval.py --batch smoke
curl -s http://localhost/api/rag/ask -d "{\"question\":\"        \"}"
```

```
./deploy/scripts/version_filter_test.sh
./deploy/scripts/permission_test.sh
```

验收项	验证内容	通过标准
上传与索引	文档上传 → 分块 → 向量化 → ES	2 分钟内 doc_chunk 可见，chunk_count 一致
混合检索	关键词 + 向量 + 版本过滤	Top-5 含标准依据文档，过期制度不出现
RAG 问答	检索 → Rerank → 生成 → 引用	回答带至少 1 条可核验引用
版本过滤	ACTIVE、SUPERSEDED、ABOLISHED	废止制度命中率为 0
权限	学生访问管理员接口	返回 403
降级	Python 运行时停止	Java 返回纯检索结果，不返回 500
评测	批量运行测试集	输出 Top5、正确率、引用一致性和过期误用率

9.2 提交代码前检查

☐ Java 服务和 Python RAG 运行时的环境变量没有写入仓库。

☐ Elasticsearch 的 embedding 维度为 768，索引字段为 dense_vector。

☐ 文档版本 status、effective_date、expiry_date 参与检索过滤。

☐ 回答引用能展开到文件名、条款位置和原文片段。

☐ 低置信度、无 citations、敏感信息和 Prompt 注入会触发拒答或拦截。

☐ 前端引用卡片能够解析 [C:n] 标记并展示来源。

附录 A 常见问题排查

现象	可能原因	处理方式
向量检索为空	embedding 维度不一致或向量未写入	确认维度 768，重新执行 reindex 并检查 ES mapping
关键词命中但向量不命中	编号、日期等精确词不适合纯语义检索	使用关键词 + 向量 RRF 混合，不要只用向量路
引用与原文不符	Prompt 约束弱或 [C:n] 解析失败	强化引用 Prompt，运行忠实度校验，不通过则降置信
废止制度仍被回答	status 更新或版本过滤条件缺失	检查 doc_document.status 和 effective/expiry 过滤
上传后没有 ES 索引	RocketMQ 未消费或消费失败	查看 search-service 日志、死信队列和手动 reindex
LLM 超时或限流	模型服务抖动或并发过高	启用 Sentinel 熔断，降级为纯检索结果
IK 分词把制度编号切碎	中文分词词典缺少制度编号	增加 IK 自定义词典，例如奖学金编号词
前端引用卡片不显示	没有解析 [C:n] 标记	用正则提取标记并映射 citations
跨域/CORS 报错	网关或 Nginx 未配置跨域和代理	配置 CorsWebFilter、Nginx /api 反代和 WebSocket 头

本手册只对应校园级 RAG 知识库平台。项目仓库中的 pom.xml、pyproject.toml、前端配置、ES mapping、Compose 文件或模型地址发生变化后，应由项目负责人统一确认并同步维护本手册。